

Migration Guide

Pydantic V2 introduces a number of changes to the API, including some breaking changes.

This page provides a guide highlighting the most important changes to help you migrate your code from Pydantic V1 to Pydantic V2.

Install Pydantic V2

Pydantic V2 is now the current production release of Pydantic. You can install Pydantic V2 from PyPI:

Terminal

```
pip install -U pydantic
```

If you encounter any issues, please [create an issue in GitHub](#) using the bug V2 label. This will help us to actively monitor and track errors, and to continue to improve the library's performance.

If you need to use latest Pydantic V1 for any reason, see the [Continue using Pydantic V1 features](#) section below for details on installation and imports from `pydantic.v1`.

Code transformation tool

We have created a tool to help you migrate your code. This tool is still in beta, but we hope it will help you to migrate your code more quickly.

You can install the tool from PyPI:

Terminal

```
pip install bump-pydantic
```

The usage is simple. If your project structure is:

- `repo_folder`

- `my_package`
 - `<python source files> ...`

Then you'll want to do:

```
cd /path/to/repo_folder bump-pydantic my_package
```

See more about it on the [Bump Pydantic](#) repository.

Continue using Pydantic V1 features

Pydantic V1 is still available when you need it, though we recommend migrating to Pydantic V2 for its improvements and new features.

If you need to use latest Pydantic V1, you can install it with:

Terminal

```
pip install "pydantic==1.*"
```

The Pydantic V2 package also continues to provide access to the Pydantic V1 API by importing through `pydantic.v1`.

For example, you can use the `BaseModel` class from Pydantic V1 instead of the Pydantic V2 `pydantic.BaseModel` class:

```
from pydantic.v1 import BaseModel
```

You can also import functions that have been removed from Pydantic V2, such as `lenient_instance`:

```
from pydantic.v1.utils import lenient_instance
```

Pydantic V1 documentation is available at <https://docs.pydantic.dev/1.10/>.

Using Pydantic v1 features in a v1/v2 environment

As of `pydantic>=1.10.17`, the `pydantic.v1` namespace can be used within V1. This makes it easier to migrate to V2, which also supports the `pydantic.v1` namespace. In order to unpin a `pydantic<2` dependency and continue using V1 features, take the following steps:

1. Replace `pydantic<2` with `pydantic>=1.10.17`
2. Find and replace all occurrences of:

```
from pydantic.<module> import <object>
```

with:

```
from pydantic.v1.<module> import <object>
```

Here's how you can import pydantic's v1 features based on your version of pydantic:

<code>pydantic>=1.10.17,<3</code>	<code>pydantic<3</code>
---	----------------------------

As of `v1.10.17` the `.v1` namespace is available in V1, allowing imports as below:

```
from pydantic.v1.fields import ModelField
```

Note

When importing modules using `pydantic>=1.10.17,<2` with the `.v1` namespace these modules will *not* be the same module as the same import without the `.v1` namespace, but the symbols imported *will* be. For example `pydantic.v1.fields` is not `pydantic.fields` but `pydantic.v1.fields.ModelField` is `pydantic.fields.ModelField`. Luckily, this is not likely to be relevant in the vast majority of cases. It's just an unfortunate consequence of providing a smoother migration experience.

Migration guide

The following sections provide details on the most important changes in Pydantic V2.

Changes to `pydantic.BaseModel`

Various method names have been changed; all non-deprecated `BaseModel` methods now have names matching either the format `model_.*` or `__.*pydantic.*__`. Where possible, we have retained the deprecated methods with their old names to help ease migration, but calling them will emit `DeprecationWarnings`.

Pydantic V1

Pydantic V2

Pydantic V1	Pydantic V2
<code>__fields__</code>	<code>model_fields</code>
<code>__private_attributes__</code>	<code>__pydantic_private__</code>
<code>__validators__</code>	<code>__pydantic_validator__</code>
<code>construct()</code>	<code>model_construct()</code>
<code>copy()</code>	<code>model_copy()</code>
<code>dict()</code>	<code>model_dump()</code>
<code>json_schema()</code>	<code>model_json_schema()</code>
<code>json()</code>	<code>model_dump_json()</code>
<code>parse_obj()</code>	<code>model_validate()</code>
<code>update_forward_refs()</code>	<code>model_rebuild()</code>

- Some of the built-in data-loading functionality has been slated for removal. In particular, `parse_raw` and `parse_file` are now deprecated. In Pydantic V2, `model_validate_json` works like `parse_raw`. Otherwise, you should load the data and then pass it to `model_validate`.
- The `from_orm` method has been deprecated; you can now just use `model_validate` (equivalent to `parse_obj` from Pydantic V1) to achieve something similar, as long as you've set `from_attributes=True` in the model config.
- The `__eq__` method has changed for models.
 - Models can only be equal to other `BaseModel` instances.
 - For two model instances to be equal, they must have the same:
 - Type (or, in the case of generic models, non-parametrized generic origin type)
 - Field values
 - Extra values (only relevant when `model_config['extra'] == 'allow'`)
 - Private attribute values; models with different values of private attributes are no

longer equal.

- Models are no longer equal to the dicts containing their data.
 - Non-generic models of different types are never equal.
 - Generic models with different origin types are never equal. We don't require *exact* type equality so that, for example, instances of `MyGenericModel[Any]` could be equal to instances of `MyGenericModel[int]`.
- We have replaced the use of the `__root__` field to specify a "custom root model" with a new type called [RootModel](#) which is intended to replace the functionality of using a field called `__root__` in Pydantic V1. Note, `RootModel` types no longer support the `arbitrary_types_allowed` config setting. See [this issue comment](#) for an explanation.
 - We have significantly expanded Pydantic's capabilities related to customizing serialization. In particular, we have added the [@field_serializer](#), [@model_serializer](#), and [@computed_field](#) decorators, which each address various shortcomings from Pydantic V1.
 - See [Custom serializers](#) for the usage docs of these new decorators.
 - Due to performance overhead and implementation complexity, we have now deprecated support for specifying `json_encoders` in the model config. This functionality was originally added for the purpose of achieving custom serialization logic, and we think the new serialization decorators are a better choice in most common scenarios.
 - We have changed the behavior related to serializing subclasses of models when they occur as nested fields in a parent model. In V1, we would always include all fields from the subclass instance. In V2, when we dump a model, we only include the fields that are defined on the annotated type of the field. This helps prevent some accidental security bugs. You can read more about this (including how to opt out of this behavior) in the [relevant section](#) of the model exporting docs.
 - `GetterDict` has been removed as it was just an implementation detail of `orm_mode`, which has been removed.
 - In many cases, arguments passed to the constructor will be **copied** in order to perform validation and, where necessary, coercion (see the [documentation](#)). This is notable in the case of passing mutable objects as arguments to a constructor.
 - The `.json()` method is deprecated, and attempting to use this deprecated method with

arguments such as `indent` or `ensure_ascii` may lead to confusing errors. For best results, switch to V2's equivalent, `model_dump_json()`. If you'd still like to use said arguments, you can use [this workaround](#).

- JSON serialization of non-string key values is generally done with `str(key)`, leading to some changes in behavior such as the following:

```
from typing import Optional

from pydantic import BaseModel as V2BaseModel
from pydantic.v1 import BaseModel as V1BaseModel


class V1Model(V1BaseModel):
    a: dict[Optional[str], int]

class V2Model(V2BaseModel):
    a: dict[Optional[str], int]


v1_model = V1Model(a={None: 123})
v2_model = V2Model(a={None: 123})

# V1
print(v1_model.json())
#> {"a": {"null": 123}}

# V2
print(v2_model.model_dump_json())
#> {"a":{"None":123}}
```

- `model_dump_json()` results are compacted in order to save space, and don't always exactly match that of `json.dumps()` output. That being said, you can easily modify the separators used in `json.dumps()` results in order to align the two outputs:

```
import json

from pydantic import BaseModel as V2BaseModel
from pydantic.v1 import BaseModel as V1BaseModel

class V1Model(V1BaseModel):
    a: list[str]

class V2Model(V2BaseModel):
    a: list[str]

v1_model = V1Model(a=['fancy', 'sushi'])
v2_model = V2Model(a=['fancy', 'sushi'])

# V1
print(v1_model.json())
#> {"a": ["fancy", "sushi"]}

# V2
print(v2_model.model_dump_json())
#> {"a":["fancy","sushi"]}

# Plain json.dumps
print(json.dumps(v2_model.model_dump()))
#> {"a": ["fancy", "sushi"]}

# Modified json.dumps
print(json.dumps(v2_model.model_dump(), separators=(',', ':')))
#> {"a":["fancy","sushi"]}
```

Changes to `pydantic.generics.GenericModel`

The `pydantic.generics.GenericModel` class is no longer necessary, and has been removed. Instead, you can now create generic `BaseModel` subclasses by just adding `Generic` as a parent class on a `BaseModel` subclass directly. This looks like `class MyGenericModel(BaseModel, Generic[T]): ...`

Mixing of V1 and V2 models is not supported which means that type parameters of such generic `BaseModel` (V2) cannot be V1 models.

While it may not raise an error, we strongly advise against using *parametrized* generics in

`isinstance` checks.

- For example, you should not do `isinstance(my_model, MyGenericModel[int])`. However, it is fine to do `isinstance(my_model, MyGenericModel)`. (Note that for standard generics, it would raise an error to do a subclass check with a parameterized generic.)
- If you need to perform `isinstance` checks against parametrized generics, you can do this by subclassing the parametrized generic class. This looks like `class MyIntModel(MyGenericModel[int]): ...` and `isinstance(my_model, MyIntModel)`.

Find more information in the [Generic models](#) documentation.

Changes to `pydantic.Field`

`Field` no longer supports arbitrary keyword arguments to be added to the JSON schema.

Instead, any extra data you want to add to the JSON schema should be passed as a dictionary to the `json_schema_extra` keyword argument.

In Pydantic V1, the `alias` property returns the field's name when no alias is set. In Pydantic V2, this behavior has changed to return `None` when no alias is set.

The following properties have been removed from or changed in `Field`:

- `const`
- `min_items` (use `min_length` instead)
- `max_items` (use `max_length` instead)
- `unique_items`
- `allow_mutation` (use `frozen` instead)
- `regex` (use `pattern` instead)
- `final` (use the [typing.Final](#) type hint instead)

`Field` constraints are no longer automatically pushed down to the parameters of generics. For example, you can no longer validate every element of a list matches a regex by providing `my_list: list[str] = Field(pattern=".*")`. Instead, use [typing.Annotated](#) to provide

an annotation on the `str` itself: `my_list: list[Annotated[str, Field(pattern=".*")]]`

Changes to dataclasses

Pydantic [dataclasses](#) continue to be useful for enabling the data validation on standard dataclasses without having to subclass `BaseModel`. Pydantic V2 introduces the following changes to this dataclass behavior:

- When used as fields, dataclasses (Pydantic or vanilla) no longer accept tuples as validation inputs; dicts should be used instead.
- The `__post_init__` in Pydantic dataclasses will now be called *after* validation, rather than before.
 - As a result, the `__post_init_post_parse__` method would have become redundant, so has been removed.
- Pydantic no longer supports `extra='allow'` for Pydantic dataclasses, where extra fields passed to the initializer would be stored as extra attributes on the dataclass. `extra='ignore'` is still supported for the purpose of ignoring unexpected fields while parsing data, they just won't be stored on the instance.
- Pydantic dataclasses no longer have an attribute `__pydantic_model__`, and no longer use an underlying `BaseModel` to perform validation or provide other functionality.
 - To perform validation, generate a JSON schema, or make use of any other functionality that may have required `__pydantic_model__` in V1, you should now wrap the dataclass with a [TypeAdapter](#) ([discussed more below](#)) and make use of its methods.
- In Pydantic V1, if you used a vanilla (i.e., non-Pydantic) dataclass as a field, the config of the parent type would be used as though it was the config for the dataclass itself as well. In Pydantic V2, this is no longer the case.
 - In Pydantic V2, to override the config (like you would with `model_config` on a `BaseModel`), you can use the `config` parameter on the `@dataclass` decorator. See [Dataclass Config](#) for examples.

Changes to config

- In Pydantic V2, to specify config on a model, you should set a class attribute called

`model_config` to be a dict with the key/value pairs you want to be used as the config. The Pydantic V1 behavior to create a class called `Config` in the namespace of the parent `BaseModel` subclass is now deprecated.

- When subclassing a model, the `model_config` attribute is inherited. This is helpful in the case where you'd like to use a base class with a given configuration for many models. Note, if you inherit from multiple `BaseModel` subclasses, like `class MyModel(Model1, Model2)`, the non-default settings in the `model_config` attribute from the two models will be merged, and for any settings defined in both, those from `Model2` will override those from `Model1`.
- The following config settings have been removed:
 - `allow_mutation` — this has been removed. You should be able to use [frozen](#) equivalently (inverse of current use).
 - `error_msg_templates`
 - `fields` — this was the source of various bugs, so has been removed. You should be able to use `Annotated` on fields to modify them as desired.
 - `getter_dict` — `orm_mode` has been removed, and this implementation detail is no longer necessary.
 - `smart_union` - the default `union_mode` in Pydantic V2 is `'smart'`.
 - `underscore_attrs_are_private` — the Pydantic V2 behavior is now the same as if this was always set to `True` in Pydantic V1.
 - `json_loads`
 - `json_dumps`
 - `copy_on_model_validation`
 - `post_init_call`
- The following config settings have been renamed:
 - `allow_population_by_field_name` → `populate_by_name` (or `validate_by_name` starting in v2.11)
 - `anystr_lower` → `str_to_lower`

- `anystr_strip_whitespace` → `str_strip_whitespace`
- `anystr_upper` → `str_to_upper`
- `keep_untouched` → `ignored_types`
- `max_anystr_length` → `str_max_length`
- `min_anystr_length` → `str_min_length`
- `orm_mode` → `from_attributes`
- `schema_extra` → `json_schema_extra`
- `validate_all` → `validate_default`

See the [ConfigDict API reference](#) for more details.

Changes to validators

`@validator` and `@root_validator` are deprecated

- `@validator` has been deprecated, and should be replaced with [@field_validator](#), which provides various new features and improvements.
 - The new `@field_validator` decorator does not have the `each_item` keyword argument; validators you want to apply to items within a generic container should be added by annotating the type argument. See [validators in Annotated metadata](#) for details. This looks like `list[Annotated[int, Field(ge=0)]]`
 - Even if you keep using the deprecated `@validator` decorator, you can no longer add the `field` or `config` arguments to the signature of validator functions. If you need access to these, you'll need to migrate to `@field_validator` — see the [next section](#) for more details.
 - If you use the `always=True` keyword argument to a validator function, note that standard validators for the annotated type will *also* be applied even to defaults, not just the custom validators. For example, despite the fact that the validator below will never error, the following code raises a `ValidationError`:

Note

To avoid this, you can use the `validate_default` argument in the `Field` function. When set to `True`, it mimics the behavior of `always=True` in Pydantic v1. However, the new way of using `validate_default` is encouraged as it provides more flexibility and control.

```
from pydantic import BaseModel, validator
```

```
class Model(BaseModel):  
    x: str = 1  
  
    @validator('x', always=True)  
    @classmethod  
    def validate_x(cls, v):  
        return v
```

```
Model()
```

- `@root_validator` has been deprecated, and should be replaced with [@model_validator](#), which also provides new features and improvements. Be aware that the allowed signatures have changed (see the [relevant documentation](#)).
 - Under some circumstances (such as assignment when `model_config['validate_assignment']` is `True`), the `@model_validator` decorator will receive an instance of the model, not a dict of values. You may need to be careful to handle this case.
 - Even if you keep using the deprecated `@root_validator` decorator, due to refactors in validation logic, you can no longer run with `skip_on_failure=False` (which is the default value of this keyword argument, so must be set explicitly to `True`).

Changes to `@validator`'s allowed signatures

In Pydantic V1, functions wrapped by `@validator` could receive keyword arguments with metadata about what was being validated. Some of these arguments have been removed from `@field_validator` in Pydantic V2:

- `config`: Pydantic V2's `config` is now a dictionary instead of a class, which means this argument is no longer backwards compatible. If you need to access the configuration you

should migrate to `@field_validator` and use `info.config`.

- `field`: this argument used to be a `ModelField` object, which was a quasi-internal class that no longer exists in Pydantic V2. Most of this information can still be accessed by using the field name from `info.field_name` to index into `cls.model_fields`

```
from pydantic import BaseModel, ValidationInfo, field_validator

class Model(BaseModel):
    x: int

    @field_validator('x')
    def val_x(cls, v: int, info: ValidationInfo) -> int:
        assert info.config is not None
        print(info.config.get('title'))
        #> Model
        print(cls.model_fields[info.field_name].is_required())
        #> True
        return v

Model(x=1)
```

TypeError is no longer converted to ValidationError in validators

Previously, when raising a `TypeError` within a validator function, that error would be wrapped into a `ValidationError` and, in some cases (such as with FastAPI), these errors might be displayed to end users. This led to a variety of undesirable behavior — for example, calling a function with the wrong signature might produce a user-facing `ValidationError`.

However, in Pydantic V2, when a `TypeError` is raised in a validator, it is no longer converted into a `ValidationError`:

```
import pytest

from pydantic import BaseModel, field_validator

class Model(BaseModel):
    x: int

    @field_validator('x')
    def val_x(cls, v: int) -> int:
        return str.lower(v) # raises a TypeError

with pytest.raises(TypeError):
    Model(x=1)
```



This applies to all validation decorators.

Validator behavior changes

Pydantic V2 includes some changes to type coercion. For example:

- coercing `int`, `float`, and `Decimal` values to strings is now optional and disabled by default, see [Coerce Numbers to Strings](#).
- iterable of pairs is no longer coerced to a dict.

See the [Conversion table](#) for details on Pydantic V2 type coercion defaults.

The `allow_reuse` keyword argument is no longer necessary

Previously, Pydantic tracked “reused” functions in decorators as this was a common source of mistakes. We did this by comparing the function’s fully qualified name (module name + function name), which could result in false positives. The `allow_reuse` keyword argument could be used to disable this when it was intentional.

Our approach to detecting repeatedly defined functions has been overhauled to only error for redefinition within a single class, reducing false positives and bringing the behavior more in line with the errors that type checkers and linters would give for defining a method with the same name multiple times in a single class definition.

In nearly all cases, if you were using `allow_reuse=True`, you should be able to simply delete that

keyword argument and have things keep working as expected.

@validate_arguments has been renamed to @validate_call

In Pydantic V2, the `@validate_arguments` decorator has been renamed to `@validate_call`.

In Pydantic V1, the decorated function had various attributes added, such as `raw_function`, and `validate` (which could be used to validate arguments without actually calling the decorated function). Due to limited use of these attributes, and performance-oriented changes in implementation, we have not preserved this functionality in `@validate_call`.

Input types are not preserved

In Pydantic V1 we made great efforts to preserve the types of all field inputs for generic collections when they were proper subtypes of the field annotations. For example, given the annotation `Mapping[str, int]` if you passed in a `collection.Counter()` you'd get a `collection.Counter()` as the value.

Supporting this behavior in V2 would have negative performance implications for the general case (we'd have to check types every time) and would add a lot of complexity to validation. Further, even in V1 this behavior was inconsistent and partially broken: it did not work for many types (`str`, `UUID`, etc.), and for generic collections it's impossible to re-build the original input correctly without a lot of special casing (consider `ChainMap`; rebuilding the input is necessary because we need to replace values after validation, e.g. if coercing strings to ints).

In Pydantic V2 we no longer attempt to preserve the input type in all cases; instead, we only promise that the output type will match the type annotations.

Going back to the `Mapping` example, we promise the output will be a valid `Mapping`, and in practice it will be a plain `dict`:

```
from collections.abc import Mapping

from pydantic import TypeAdapter

class MyDict(dict):
    pass

ta = TypeAdapter(Mapping[str, int])
v = ta.validate_python(MyDict())
print(type(v))
#> <class 'dict'>
```



If you want the output type to be a specific type, consider annotating it as such or implementing a custom validator:

```
from collections.abc import Mapping
from typing import Annotated, Any, TypeVar

from pydantic import (
    TypeAdapter,
    ValidationInfo,
    ValidatorFunctionWrapHandler,
    WrapValidator,
)

def restore_input_type(
    value: Any, handler: ValidatorFunctionWrapHandler, _info: ValidationInfo
) -> Any:
    return type(value)(handler(value))

T = TypeVar('T')
PreserveType = Annotated[T, WrapValidator(restore_input_type)]

ta = TypeAdapter(PreserveType[Mapping[str, int]])

class MyDict(dict):
    pass

v = ta.validate_python(MyDict())
assert type(v) is MyDict
```



While we don't promise to preserve input types everywhere, we *do* preserve them for subclasses of `BaseModel`, and for dataclasses:

```
import pydantic.dataclasses
from pydantic import BaseModel

class InnerModel(BaseModel):
    x: int

class OuterModel(BaseModel):
    inner: InnerModel

class SubInnerModel(InnerModel):
    y: int

m = OuterModel(inner=SubInnerModel(x=1, y=2))
print(m)
#> inner=SubInnerModel(x=1, y=2)

@pydantic.dataclasses.dataclass
class InnerDataclass:
    x: int

@pydantic.dataclasses.dataclass
class SubInnerDataclass(InnerDataclass):
    y: int

@pydantic.dataclasses.dataclass
class OuterDataclass:
    inner: InnerDataclass

d = OuterDataclass(inner=SubInnerDataclass(x=1, y=2))
print(d)
#> OuterDataclass(inner=SubInnerDataclass(x=1, y=2))
```

Changes to Handling of Standard Types

Dicts

Iterables of pairs (which include empty iterables) no longer pass validation for fields of type `dict`.

Unions

While union types will still attempt validation of each choice from left to right, they now preserve the type of the input whenever possible, even if the correct type is not the first choice for which the input would pass validation. As a demonstration, consider the following example:

```
from typing import Union

from pydantic import BaseModel

class Model(BaseModel):
    x: Union[int, str]

print(Model(x='1'))
#> x='1'
```



In Pydantic V1, the printed result would have been `x=1`, since the value would pass validation as an `int`. In Pydantic V2, we recognize that the value is an instance of one of the cases and short-circuit the standard union validation.

To revert to the non-short-circuiting left-to-right behavior of V1, annotate the union with `Field(union_mode='left_to_right')`. See [Union Mode](#) for more details.

Required, optional, and nullable fields

Pydantic V2 changes some of the logic for specifying whether a field annotated as `Optional` is required (i.e., has no default value) or not (i.e., has a default value of `None` or any other value of the corresponding type), and now more closely matches the behavior of `dataclasses`. Similarly, fields annotated as `Any` no longer have a default value of `None`.

The following table describes the behavior of field annotations in V2:

State	Field Definition
Required, cannot be None	<code>f1: str</code>

State	Field Definition
Not required, cannot be None, is 'abc ' by default	f2: str = 'abc '
Required, can be None	f3: Optional[str]
Not required, can be None, is None by default	f4: Optional[str] = None
Not required, can be None, is 'abc ' by default	f5: Optional[str] = 'abc '
Required, can be any type (including None)	f6: Any
Not required, can be any type (including None)	f7: Any = None

 Note

A field annotated as `typing.Optional[T]` will be required, and will allow for a value of `None`. It does not mean that the field has a default value of `None`. *(This is a breaking change from V1.)*

 Note

Any default value if provided makes a field not required.

Here is a code example demonstrating the above:

```
from typing import Optional

from pydantic import BaseModel, ValidationError

class Foo(BaseModel):
    f1: str # required, cannot be None
    f2: Optional[str] # required, can be None - same as str | None
    f3: Optional[str] = None # not required, can be None
    f4: str = 'Foobar' # not required, but cannot be None

try:
    Foo(f1=None, f2=None, f4='b')
except ValidationError as e:
    print(e)
    """
    1 validation error for Foo
    f1
      Input should be a valid string [type=string_type, input_value=None, input_type=None]
    """
```

Patterns / regex on strings

Pydantic V1 used Python's regex library. Pydantic V2 uses the Rust [regex crate](#). This crate is not just a "Rust version of regular expressions", it's a completely different approach to regular expressions. In particular, it promises linear time searching of strings in exchange for dropping a couple of features (namely look arounds and backreferences). We believe this is a tradeoff worth making, in particular because Pydantic is used to validate untrusted input where ensuring things don't accidentally run in exponential time depending on the untrusted input is important. On the flipside, for anyone not using these features complex regex validation should be orders of magnitude faster because it's done in Rust and in linear time.

If you still want to use Python's regex library, you can use the [regex_engine](#) config setting.

Type conversion from floats to integers

In V1, whenever a field was annotated as `int`, any float value would be accepted, which could lead to a potential data loss if the float value contains a non-zero decimal part. In V2, type conversion from floats to integers is only allowed if the decimal part is zero:

```
from pydantic import BaseModel, ValidationError

class Model(BaseModel):
    x: int

print(Model(x=10.0))
#> x=10
try:
    Model(x=10.2)
except ValidationError as err:
    print(err)
    """
    1 validation error for Model
    x
      Input should be a valid integer, got a number with a fractional part [type=int_fr
```

Introduction of TypeAdapter

Pydantic V1 had weak support for validating or serializing non-BaseModel types.

To work with them, you had to either create a “root” model or use the utility functions in `pydantic.tools` (namely, `parse_obj_as` and `schema_of`).

In Pydantic V2 this is *a lot* easier: the [TypeAdapter](#) class lets you create an object with methods for validating, serializing, and producing JSON schemas for arbitrary types. This serves as a complete replacement for `parse_obj_as` and `schema_of` (which are now deprecated), and also covers some of the use cases of “root” models. ([RootModel](#), [discussed above](#), covers the others.)

```
from pydantic import TypeAdapter

adapter = TypeAdapter(list[int])
assert adapter.validate_python(['1', '2', '3']) == [1, 2, 3]
print(adapter.json_schema())
#> {'items': {'type': 'integer'}, 'type': 'array'}
```

Due to limitations of inferring generic types with common type checkers, to get proper typing in some scenarios, you may need to explicitly specify the generic parameter:

```
from pydantic import TypeAdapter

adapter = TypeAdapter[str | int](str | int)
...
```



See [Type Adapter](#) for more information.

Defining custom types

We have completely overhauled the way custom types are defined in pydantic.

We have exposed hooks for generating both pydantic-core and JSON schemas, allowing you to get all the performance benefits of Pydantic V2 even when using your own custom types.

We have also introduced ways to use [typing.Annotated](#) to add custom validation to your own types.

The main changes are:

- `__get_validators__` should be replaced with `__get_pydantic_core_schema__`. See [Custom Data Types](#) for more information.
- `__modify_schema__` becomes `__get_pydantic_json_schema__`. See [JSON Schema Customization](#) for more information.

Additionally, you can use [typing.Annotated](#) to modify or provide the `__get_pydantic_core_schema__` and `__get_pydantic_json_schema__` functions of a type by annotating it, rather than modifying the type itself. This provides a powerful and flexible mechanism for integrating third-party types with Pydantic, and in some cases may help you remove hacks from Pydantic V1 introduced to work around the limitations for custom types.

See [Custom Data Types](#) for more information.

Changes to JSON schema generation

We received many requests over the years to make changes to the JSON schemas that pydantic generates.

In Pydantic V2, we have tried to address many of the common requests:

- The JSON schema for `Optional` fields now indicates that the value `null` is allowed.
- The `Decimal` type is now exposed in JSON schema (and serialized) as a string.
- The JSON schema no longer preserves `namedtuples` as `namedtuples`.
- The JSON schema we generate by default now targets draft 2020-12 (with some OpenAPI extensions).
- When they differ, you can now specify if you want the JSON schema representing the inputs to validation, or the outputs from serialization.

However, there have been many reasonable requests over the years for changes which we have not chosen to implement.

In Pydantic V1, even if you were willing to implement changes yourself, it was very difficult because the JSON schema generation process involved various recursive function calls; to override one, you'd have to copy and modify the whole implementation.

In Pydantic V2, one of our design goals was to make it easier to customize JSON schema generation. To this end, we have introduced the class [GenerateJsonSchema](#), which implements the translation of a type's pydantic-core schema into a JSON schema. By design, this class breaks the JSON schema generation process into smaller methods that can be easily overridden in subclasses to modify the "global" approach to generating JSON schema.

The various methods that can be used to produce JSON schema (such as `BaseModel.model_json_schema` or `TypeAdapter.json_schema`) accept a keyword argument `schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema`, and you can pass your custom subclass to these methods in order to use your own approach to generating JSON schema.

Hopefully this means that if you disagree with any of the choices we've made, or if you are reliant on behaviors in Pydantic V1 that have changed in Pydantic V2, you can use a custom `schema_generator`, modifying the `GenerateJsonSchema` class as necessary for your application.

BaseSettings has moved to pydantic-settings

[BaseSettings](#), the base object for Pydantic [settings management](#), has been moved to a

separate package, [pydantic-settings](#).

Also, the `parse_env_var` classmethod has been removed. So, you need to [customise settings sources](#) to have your own parsing function.

Color and Payment Card Numbers moved to `pydantic-extra-types`

The following special-use types have been moved to the [Pydantic Extra Types](#) package, which may be installed separately if needed.

- [Color Types](#)
- [Payment Card Numbers](#)

Url and Dsn types in `pydantic.networks` no longer inherit from `str`

In Pydantic V1 the [AnyUrl](#) type inherited from `str`, and all the other `Url` and `Dsn` types inherited from these. In Pydantic V2 these types are built on two new `Url` and `MultiHostUrl` classes using `Annotated`.

Inheriting from `str` had upsides and downsides, and for V2 we decided it would be better to remove this. To use these types in APIs which expect `str` you'll now need to convert them (with `str(url)`).

Pydantic V2 uses Rust's [Url](#) crate for URL validation. Some of the URL validation differs slightly from the previous behavior in V1. One notable difference is that the new `Url` types append slashes to the validated version if no path is included, even if a slash is not specified in the argument to a `Url` type constructor. See the example below for this behavior:

```
from pydantic import AnyUrl

assert str(AnyUrl(url='https://google.com')) == 'https://google.com/'
assert str(AnyUrl(url='https://google.com/')) == 'https://google.com/'
assert str(AnyUrl(url='https://google.com/api')) == 'https://google.com/api/'
assert str(AnyUrl(url='https://google.com/api/')) == 'https://google.com/api/'
```

If you still want to use the old behavior without the appended slash, take a look at this [solution](#).

Constrained types

The `Constrained*` classes were *removed*, and you should replace them by `Annotated[<type>, Field(...)]`, for example:

```
from pydantic import BaseModel, ConstrainedInt

class MyInt(ConstrainedInt):
    ge = 0

class Model(BaseModel):
    x: MyInt
```

...becomes:

```
from typing import Annotated

from pydantic import BaseModel, Field

MyInt = Annotated[int, Field(ge=0)]

class Model(BaseModel):
    x: MyInt
```

Read more about it in the [Composing types via Annotated](#) docs.

For `ConstrainedStr` you can use [StringConstraints](#) instead.

Mypy plugins

Pydantic V2 contains a [mypy](#) plugin in `pydantic.mypy`.

When using [V1 features](#) the `pydantic.v1.mypy` plugin might need to also be enabled.

To configure the mypy plugins:

mypy.ini

pyproject.toml

```
[mypy]
plugins = pydantic.mypy, pydantic.v1.mypy # include `v1.mypy` if required.
```

Other changes

- Dropped support for [email-validator<2.0.0](#). Make sure to update using `pip install -U email-validator`.

Moved in Pydantic V2

Pydantic V1	Pydantic V2
<code>pydantic.BaseSettings</code>	pydantic_settings.BaseSettings
<code>pydantic.color</code>	pydantic_extra_types.color
<code>pydantic.types.PaymentCardBrand</code>	pydantic_extra_types.PaymentCardBr
<code>pydantic.types.PaymentCardNumber</code>	pydantic_extra_types.PaymentCardNu
<code>pydantic.utils.version_info</code>	pydantic.version.version_info
<code>pydantic.error_wrappers.ValidationError</code>	pydantic.ValidationError
<code>pydantic.utils.to_camel</code>	pydantic.alias_generators.to_pascal
<code>pydantic.utils.to_lower_camel</code>	pydantic.alias_generators.to_camel
<code>pydantic.PyObject</code>	pydantic.ImportString

Deprecated and moved in Pydantic V2

Pydantic V1	Pydantic V2
<code>pydantic.tools.schema_of</code>	<code>pydantic.deprecated.tools.schema_of</code>
<code>pydantic.tools.parse_obj_as</code>	<code>pydantic.deprecated.tools.parse_obj_as</code>
<code>pydantic.tools.schema_json_of</code>	<code>pydantic.deprecated.tools.schema_json_of</code>
<code>pydantic.json.pydantic_encoder</code>	<code>pydantic.deprecated.json.pydantic_encoder</code>

Pydantic V1**Pydantic V2**

<code>pydantic.validate_arguments</code>	<code>pydantic.deprecated.decorator.val</code>
<code>pydantic.json.custom_pydantic_encoder</code>	<code>pydantic.deprecated.json.custom_p</code>
<code>pydantic.json.ENCODERS_BY_TYPE</code>	<code>pydantic.deprecated.json.ENCODERS</code>
<code>pydantic.json.timedelta_isoformat</code>	<code>pydantic.deprecated.json.timedelt</code>
<code>pydantic.decorator.validate_arguments</code>	<code>pydantic.deprecated.decorator.val</code>
<code>pydantic.class_validators.validator</code>	<code>pydantic.deprecated.class_validat</code>
<code>pydantic.class_validators.root_validator</code>	<code>pydantic.deprecated.class_validat</code>
<code>pydantic.utils.deep_update</code>	<code>pydantic.v1.utils.deep_update</code>
<code>pydantic.utils.GetterDict</code>	<code>pydantic.v1.utils.GetterDict</code>
<code>pydantic.utils.lenient_issubclass</code>	<code>pydantic.v1.utils.lenient_issubcl</code>
<code>pydantic.utils.lenient_isinstance</code>	<code>pydantic.v1.utils.lenient_isinsta</code>
<code>pydantic.utils.is_valid_field</code>	<code>pydantic.v1.utils.is_valid_field</code>
<code>pydantic.utils.update_not_none</code>	<code>pydantic.v1.utils.update_not_none</code>
<code>pydantic.utils.import_string</code>	<code>pydantic.v1.utils.import_string</code>
<code>pydantic.utils.Representation</code>	<code>pydantic.v1.utils.Representation</code>
<code>pydantic.utils.ROOT_KEY</code>	<code>pydantic.v1.utils.ROOT_KEY</code>
<code>pydantic.utils.smart_deepcopy</code>	<code>pydantic.v1.utils.smart_deepcopy</code>
<code>pydantic.utils.sequence_like</code>	<code>pydantic.v1.utils.sequence_like</code>

Removed in Pydantic V2

- `pydantic.ConstrainedBytes`
- `pydantic.ConstrainedDate`
- `pydantic.ConstrainedDecimal`
- `pydantic.ConstrainedFloat`
- `pydantic.ConstrainedFrozenSet`
- `pydantic.ConstrainedInt`
- `pydantic.ConstrainedList`
- `pydantic.ConstrainedSet`
- `pydantic.ConstrainedStr`
- `pydantic.JsonWrapper`
- `pydantic.NoneBytes`
 - This was an alias to `None` | `bytes`.
- `pydantic.NoneStr`
 - This was an alias to `None` | `str`.
- `pydantic.NoneStrBytes`
 - This was an alias to `None` | `str` | `bytes`.
- `pydantic.Protocol`
- `pydantic.Required`
- `pydantic.StrBytes`
 - This was an alias to `str` | `bytes`.
- `pydantic.compiled`
- `pydantic.config.get_config`
- `pydantic.config.inherit_config`
- `pydantic.config.prepare_config`
- `pydantic.create_model_from_namedtuple`
- `pydantic.create_model_from_typeddict`
- `pydantic.dataclasses.create_pydantic_model_from_dataclass`
- `pydantic.dataclasses.make_dataclass_validator`
- `pydantic.dataclasses.set_validation`

- `pydantic.datetime_parse.parse_date`
- `pydantic.datetime_parse.parse_time`
- `pydantic.datetime_parse.parse_datetime`
- `pydantic.datetime_parse.parse_duration`
- `pydantic.error_wrappers.ErrorWrapper`
- `pydantic.errors.AnyStrMaxLengthError`
- `pydantic.errors.AnyStrMinLengthError`
- `pydantic.errors.ArbitraryTypeError`
- `pydantic.errors.BoolError`
- `pydantic.errors.BytesError`
- `pydantic.errors.CallableError`
- `pydantic.errors.ClassError`
- `pydantic.errors.ColorError`
- `pydantic.errors.ConfigError`
- `pydantic.errors.DataclassTypeError`
- `pydantic.errors.DateError`
- `pydantic.errors.DateNotInTheFutureError`
- `pydantic.errors.DateNotInThePastError`
- `pydantic.errors.DateTimeError`
- `pydantic.errors.DecimalError`
- `pydantic.errors.DecimalIsNotFiniteError`
- `pydantic.errors.DecimalMaxDigitsError`
- `pydantic.errors.DecimalMaxPlacesError`
- `pydantic.errors.DecimalWholeDigitsError`
- `pydantic.errors.DictError`
- `pydantic.errors.DurationError`
- `pydantic.errors.EmailError`
- `pydantic.errors.EnumError`

- `pydantic.errors.EnumMemberError`
- `pydantic.errors.ExtraError`
- `pydantic.errors.FloatError`
- `pydantic.errors.FrozenSetError`
- `pydantic.errors.FrozenSetMaxLengthError`
- `pydantic.errors.FrozenSetMinLengthError`
- `pydantic.errors.HashableError`
- `pydantic.errors.IPv4AddressError`
- `pydantic.errors.IPv4InterfaceError`
- `pydantic.errors.IPv4NetworkError`
- `pydantic.errors.IPv6AddressError`
- `pydantic.errors.IPv6InterfaceError`
- `pydantic.errors.IPv6NetworkError`
- `pydantic.errors.IPvAnyAddressError`
- `pydantic.errors.IPvAnyInterfaceError`
- `pydantic.errors.IPvAnyNetworkError`
- `pydantic.errors.IntEnumError`
- `pydantic.errors.IntegerError`
- `pydantic.errors.InvalidByteSize`
- `pydantic.errors.InvalidByteSizeUnit`
- `pydantic.errors.InvalidDiscriminator`
- `pydantic.errors.InvalidLengthForBrand`
- `pydantic.errors.JsonError`
- `pydantic.errors.JsonTypeError`
- `pydantic.errors.ListError`
- `pydantic.errors.ListMaxLengthError`
- `pydantic.errors.ListMinLengthError`
- `pydantic.errors.ListUniqueItemsError`

- `pydantic.errors.LuhnValidationError`
- `pydantic.errors.MissingDiscriminator`
- `pydantic.errors.MissingError`
- `pydantic.errors.NoneIsAllowedError`
- `pydantic.errors.NoneIsNotAllowedError`
- `pydantic.errors.NotDigitError`
- `pydantic.errors.NotNoneError`
- `pydantic.errors.NumberNotGeError`
- `pydantic.errors.NumberNotGtError`
- `pydantic.errors.NumberNotLeError`
- `pydantic.errors.NumberNotLtError`
- `pydantic.errors.NumberNotMultipleError`
- `pydantic.errors.PathError`
- `pydantic.errors.PathNotADirectoryError`
- `pydantic.errors.PathNotAFileError`
- `pydantic.errors.PathNotExistsError`
- `pydantic.errors.PatternError`
- `pydantic.errors.PyObjectError`
- `pydantic.errors.PydanticTypeError`
- `pydantic.errors.PydanticValueError`
- `pydantic.errors.SequenceError`
- `pydantic.errors.SetError`
- `pydantic.errors.SetMaxLengthError`
- `pydantic.errors.SetMinLengthError`
- `pydantic.errors.StrError`
- `pydantic.errors.StrRegexError`
- `pydantic.errors.StrictBoolError`
- `pydantic.errors.SubclassError`

- `pydantic.errors.TimeError`
- `pydantic.errors.TupleError`
- `pydantic.errors.TupleLengthError`
- `pydantic.errors.UUIDError`
- `pydantic.errors.UUIDVersionError`
- `pydantic.errors.UrlError`
- `pydantic.errors.UrlExtraError`
- `pydantic.errors.UrlHostError`
- `pydantic.errors.UrlHostTldError`
- `pydantic.errors.UrlPortError`
- `pydantic.errors.UrlSchemeError`
- `pydantic.errors.UrlSchemePermittedError`
- `pydantic.errors.UrlUserInfoError`
- `pydantic.errors.WrongConstantError`
- `pydantic.main.validate_model`
- `pydantic.networks.stricturl`
- `pydantic.parse_file_as`
- `pydantic.parse_raw_as`
- `pydantic.stricturl`
- `pydantic.tools.parse_file_as`
- `pydantic.tools.parse_raw_as`
- `pydantic.types.JsonWrapper`
- `pydantic.types.NoneBytes`
- `pydantic.types.NoneStr`
- `pydantic.types.NoneStrBytes`
- `pydantic.types.PyObject`
- `pydantic.types.StrBytes`
- `pydantic.typing.evaluate_forwardref`

- `pydantic.typing.AbstractSetIntStr`
- `pydantic.typing.AnyCallable`
- `pydantic.typing.AnyClassMethod`
- `pydantic.typing.CallableGenerator`
- `pydantic.typing.DictAny`
- `pydantic.typing.DictIntStrAny`
- `pydantic.typing.DictStrAny`
- `pydantic.typing.IntStr`
- `pydantic.typing.ListStr`
- `pydantic.typing.MappingIntStrAny`
- `pydantic.typing.NoArgAnyCallable`
- `pydantic.typing.NoneType`
- `pydantic.typing.ReprArgs`
- `pydantic.typing.SetStr`
- `pydantic.typing.StrPath`
- `pydantic.typing.TupleGenerator`
- `pydantic.typing.WithArgsTypes`
- `pydantic.typing.all_literal_values`
- `pydantic.typing.display_as_type`
- `pydantic.typing.get_all_type_hints`
- `pydantic.typing.get_args`
- `pydantic.typing.get_origin`
- `pydantic.typing.get_sub_types`
- `pydantic.typing.is_callable_type`
- `pydantic.typing.is_classvar`
- `pydantic.typing.is_finalvar`
- `pydantic.typing.is_literal_type`
- `pydantic.typing.is_namedtuple`

- `pydantic.typing.is_new_type`
- `pydantic.typing.is_none_type`
- `pydantic.typing.is_typeddict`
- `pydantic.typing.is_typeddict_special`
- `pydantic.typing.is_union`
- `pydantic.typing.new_type_supertype`
- `pydantic.typing.resolve_annotations`
- `pydantic.typing.typing_base`
- `pydantic.typing.update_field_forward_refs`
- `pydantic.typing.update_model_forward_refs`
- `pydantic.utils.ClassAttribute`
- `pydantic.utils.DUNDER_ATTRIBUTES`
- `pydantic.utils.PyObjectStr`
- `pydantic.utils.ValueItems`
- `pydantic.utils.almost_equal_floats`
- `pydantic.utils.get_discriminator_alias_and_values`
- `pydantic.utils.get_model`
- `pydantic.utils.get_unique_discriminator_alias`
- `pydantic.utils.in_ipython`
- `pydantic.utils.is_valid_identifier`
- `pydantic.utils.path_type`
- `pydantic.utils.validate_field_name`
- `pydantic.validate_model`

Was this page helpful?



© Pydantic Services Inc. 2025 to present

